Αναφορά προόδου: προσομοιωτής Sim v2.1

Πειράματα της 17ης Οκτωβρίου 2026

QRL-Lab

Κώστα, εδώ είναι η αναφορά με όσα έγιναν σήμερα, βήμα βήμα και με απλά λόγια. Ακολουθεί τη δομή του Evaluation του DInos paper σου.

Διερευνητικά αποτελέσματα. Τα gates (preflight και rev-1) απέτυχαν και ο προσομοιωτής δεν είναι βαθμονομημένος. Τα νούμερα δεν είναι αποτέλεσμα για paper. Κανένας RL agent δεν έχει εκπαιδευτεί σε αυτά τα runs.

---

1 Τι θέλαμε να ελέγξουμε

Θέλουμε να συγκρίνουμε, σε έναν προσομοιωτή, έναν κβαντικό RL agent με τους κλασικούς agents (DERP, DInos). Πριν από αυτό χρειαζόμαστε μια απλή απόδειξη ότι ο προσομοιωτής είναι λογικός. Αν ένας απλός ελεγκτής με κατώφλι (threshold) δεν κερδίζει ούτε έναν ελεγκτή που κρατά σταθερό αριθμό pods, κάτι δεν πάει καλά με το περιβάλλον. Τότε οι συγκρίσεις agents δεν θα έλεγαν τίποτα. Αυτή είναι η ερώτηση της αναφοράς: περνά ο προσομοιωτής τους ελέγχους λογικής, ώστε να χτίσουμε πάνω του;

2 Τι αλλάξαμε

- **Sim v2 → v2.1.** Προσθέσαμε ρυθμιστικά «κουμπιά» (knobs) για να δούμε πόσο εξαρτώνται τα αποτελέσματα από παραμέτρους που δεν έχουμε μετρήσει: `latency_add`, `tail_scale`, `cpu_offset`, rebalance k/delta και theta. Οι τιμές τους είναι παγωμένες στο `code/sim_v2/sensitivity_v21.json`. Με τις default τιμές το v2.1 δίνει bit-identical αποτελέσματα με το v2, δηλαδή ίδια ως το τελευταίο ψηφίο.
- **Παγωμένες ρυθμίσεις (freeze).** Πριν τρέξει ένα πείραμα, καταγράφουμε τον κώδικα (git) και τις ρυθμίσεις (hash, δηλαδή ένα «δακτυλικό αποτύπωμα» αρχείου). Έτσι δεν μπορούμε να τις αλλάξουμε αφού δούμε τα αποτελέσματα.
- **Χωρισμός seeds.** Seed είναι ο αριθμός που καθορίζει την τυχαιότητα ενός επεισοδίου. Χρησιμοποιούμε χωριστά σύνολα. Τα 1000–1004 είναι για ρύθμιση (tuning). Τα 1200–1209 είναι φρέσκα, για το rev-1 gate. Τα 20000+ μένουν ανέγγιχτα για την τελική αξιολόγηση. Τα 1100–1109 «κάηκαν» από ένα smoke test και δεν χρησιμοποιούνται.
- **Πιστός K8s HPA.** Το HPA είναι ο αυτόματος scaler του Kubernetes. Γράψαμε τον baseline μας από τον πραγματικό κώδικα του Kubernetes v1.34.0 (`docs/hpa_spec.md`): υπολογίζει τα pods ως το (τωρινά pods × CPU / στόχος) στρογγυλεμένο προς τα πάνω, αγνοεί αλλαγές μέσα σε ανοχή 10%, ανεβαίνει γρήγορα (έως 100% ή 4 pods ανά 15 s) και κατεβαίνει αργά (παράθυρο σταθεροποίησης 300 s, δηλαδή 20 βήματα).
- **Ένα συγκεκριμένο όριο του HPA.** Στον προσομοιωτή δεν έχουμε πραγματική μέτρηση CPU ως προς το request του pod. Χρησιμοποιούμε ως υποκατάστατο το ποσοστό

«απασχόλησης» (served/capacity, μέγιστο 100%) και υποθέτουμε request = limit. Αυτό είναι υπόθεση. Το υποκατάστατο δεν ξεχωρίζει μέτρια από ακραία υπερφόρτωση.

3 Πειραματική διάταξη

Περιβάλλον. Όπως στο DInos, ένα βήμα ελέγχου είναι 15 δευτερόλεπτα και ένα επεισόδιο έχει 5000 βήματα. Ο προσομοιωτής παίρνει φορτίο (αιτήματα ανά δευτερόλεπτο), έχει χωρητικότητα ανάλογη με τον αριθμό των Redis pods, και υπολογίζει CPU και latency. Ένα νέο pod χρειάζεται μερικά βήματα για να ξεκινήσει. Το φορτίο που χρησιμοποιούμε λέγεται W0: ημίτονο, θόρυβος και ένα spike διάρκειας 100 βημάτων σε τυχαία θέση. Μνήμη δεν μοντελοποιείται. Όλες οι παράμετροι είναι προσωρινές.

Μετρική. Το κύριο μέτρο είναι το συνολικό reward ανά επεισόδιο (άθροισμα στα 5000 βήματα). Εδώ το reward είναι κόστος, άρα είναι αρνητικό: ποινή για κάθε pod, ποινή όταν παραβιάζεται το SLA (το όριο latency) και ποινή για αλλαγές στον αριθμό pods. Πιο κοντά στο μηδέν σημαίνει καλύτερα. Αναφέρουμε επίσης μέσο CPU, μέσο p95 latency (η τιμή κάτω από την οποία πέφτει το 95% των αιτημάτων) και μέσο αριθμό pods.

Baselines (απλοί ελεγκτές για σύγκριση):

- **threshold 40/25/0**: αντιδραστικός κανόνας. Όταν το φορτίο περάσει ένα πάνω κατώφλι προσθέτει pods, όταν πέσει κάτω από ένα κάτω κατώφλι αφαιρεί. Οι τιμές 40/25/0 επιλέχθηκαν στο preflight.
- **fixed(22)** (στα σχήματα και `fixed_best`): σταθερά 22 pods, προ-παρασχεμένα, δηλαδή έτοιμα από την αρχή. Το 22 ήταν το καλύτερο σταθερό νούμερο στα seeds 1000–1004.
- **static_ramp(22)**: ξεκινά από 6 pods όπως όλοι οι άλλοι, ανεβαίνει ένα pod ανά βήμα μέχρι το 22 και μένει εκεί. Είναι το δίκαιο αντίστοιχο του fixed, γιατί δεν έχει προβάδισμα εκκίνησης.
- **always_min / always_max**: πάντα το ελάχιστο ή το μέγιστο των pods. Είναι έλεγχοι λογικής.
- **K8sHPA**: ο scaler του Kubernetes όπως περιγράφηκε πιο πάνω, με στόχο CPU 40%.
- **oracle** (clairvoyant): «ξέρει το μέλλον». Λύνει το πρόβλημα γνωρίζοντας το φορτίο των επόμενων βημάτων και δίνει ένα όριο για το πόσο καλά μπορεί να γίνει με κινήσεις ±1 pod. Δεν μπορεί να εφαρμοστεί σε πραγματικό σύστημα και δεν είναι άνω όριο για πολυ-pod ελεγκτές όπως το HPA.

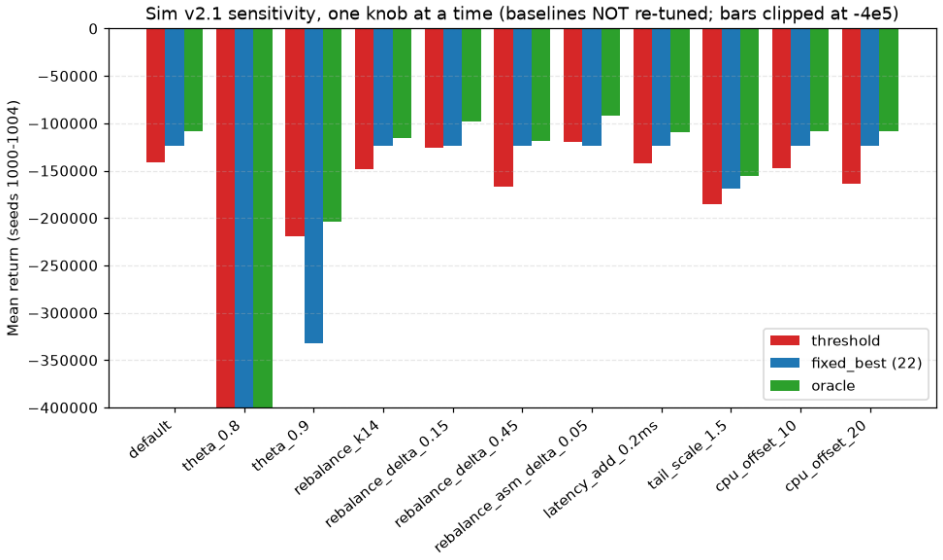
4 Βήμα-βήμα τα πειράματα

4.1 Βήμα 1. Preflight (R-20261017-004)

Ερώτηση: κερδίζει ο tuned threshold έναν σταθερό αριθμό pods; **Τρέξαμε:** threshold εναντίον fixed(22), 5 seeds (1000–1004). **Βγήκε:** threshold −140.9k, fixed(22) −123.3k, νίκες του threshold 0/5. Το gate **απέτυχε**. **Τι σημαίνει:** ο threshold έχασε από έναν ελεγκτή που δεν κάνει τίποτα έξυπνο. Αυτό είναι ύποπτο. Δύο εξηγήσεις: ο προσομοιωτής δεν είναι ρεαλιστικός, ή η σύγκριση είναι άδικη επειδή το fixed ξεκινά με 22 έτοιμα pods και ο threshold με 6.

4.2 Βήμα 2. Διάγνωση (R-20261017-010)

Ερώτηση: από πού προέρχεται η διαφορά; **Τρέξαμε:** traces ανά βήμα στα ίδια 5 seeds, χωρίς re-tuning, με χωριστή λογιστική για την εκκίνηση (πρώτα 150 βήματα) και τα υπόλοιπα, και δοκιμές ευαισθησίας σε 10 configs. **Βγήκε:** στην εκκίνηση ο threshold χάνει −18.4k και το fixed −3.5k. Άρα ένα μέρος της διαφοράς είναι το προβάδισμα εκκίνησης. Στις δοκιμές ευαισθησίας η κατάταξη αλλάζει: με ϑ=0.9 threshold −219.5k και fixed −332.7k, ενώ με rebalance_asm_delta_0.05 threshold −119.4k και fixed −123.3k (το ϑ=0.8 βγήκε εκτός κλίμακας και κόπηκε στο −4e5). **Τι σημαίνει:** η κατάταξη εξαρτάται από παραμέτρους που δεν έχουμε μετρήσει, και αυτό δείχνει ότι χρειάζεται βαθμονόμηση.



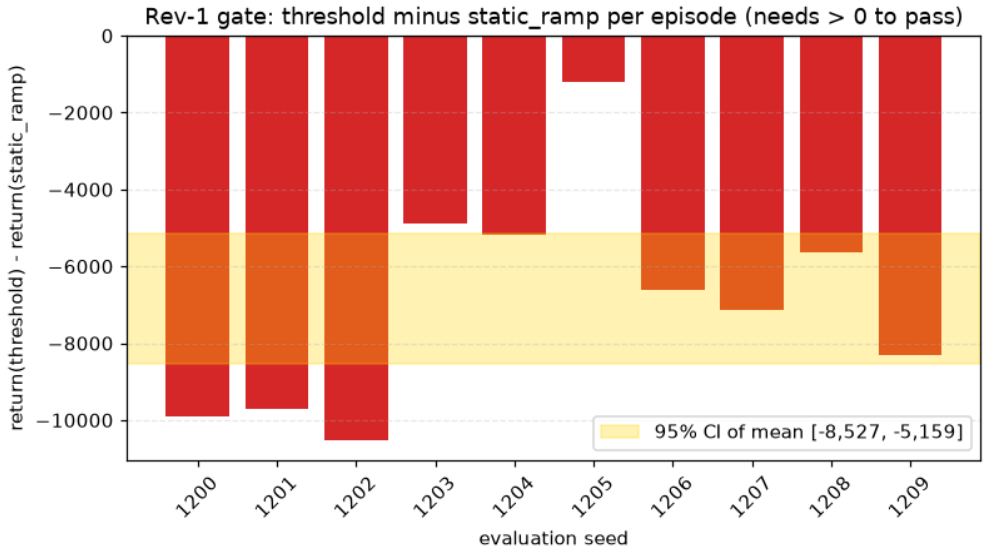
Σχήμα Α. Ευαισθησία του προσομοιωτή: μέσο return ανά config, ένα knob τη φορά. Πηγή: `20261017-145935-R-20261017-010-e0-a1/diagnose.json`.

4.3 Βήμα 3. Rev-1 gate (R-20261017-013)

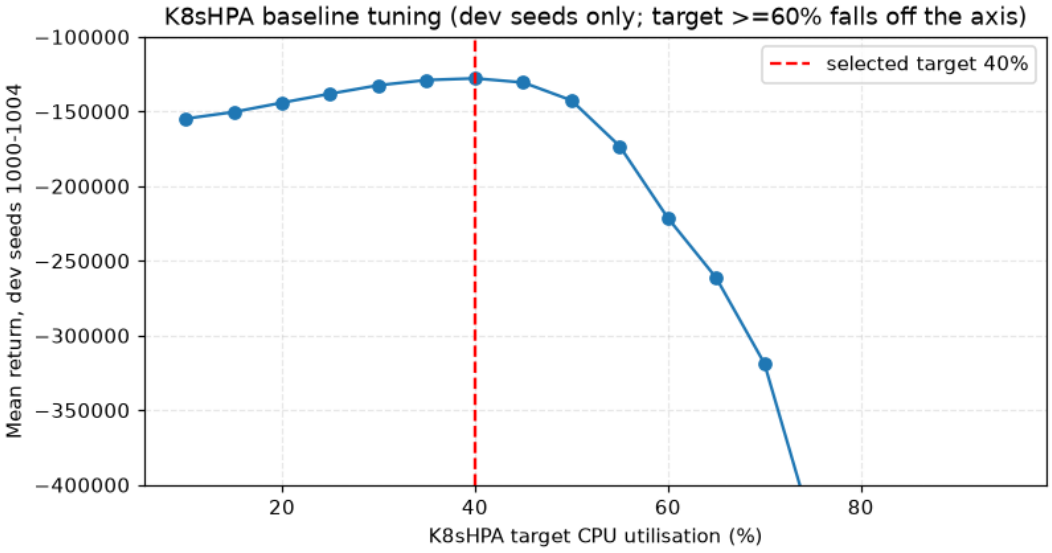
Ερώτηση: αν όλοι ξεκινούν από τα ίδια 6 pods, κερδίζει ο threshold; **Τρέξαμε:** c0=6, 10 φρέσκα seeds (1200–1209), threshold εναντίον static_ramp(22), μαζί με fixed_best, always_min/max και oracle. Οι κανόνες γράφτηκαν πριν το run (`PREREG_rev1.md`) και ο έλεγχος περνά μόνο αν το 95% CI της ζευγαρωτής διαφοράς είναι πάνω από το 0. **Βγήκε:** threshold −133.1k, static_ramp −126.2k, διαφορά −6.9k, 95% CI [−8.5k, −5.2k], νίκες 0/10. Επίσης oracle −101.6k, fixed_best −120.8k, always_max −157.2k, always_min −3.55M. Το gate **απέτυχε**. **Τι σημαίνει:** ακόμα και με ίδια εκκίνηση ο threshold χάνει, σε κάθε seed. Το πρόβλημα δεν εξηγείται μόνο από την εκκίνηση. Το πρώτο FAIL παραμένει καταγεγραμμένο μαζί με το δεύτερο.

4.4 Βήμα 4. Ρύθμιση του K8sHPA

Ερώτηση: ποιος στόχος CPU δίνει το καλύτερο HPA; **Τρέξαμε:** `select_hpa_dev.py`, στόχοι 10% έως 95% ανά 5, στα seeds ανάπτυξης 1000–1004 με default config. Δεν είναι ξεχωριστό run με id στη βάση. **Βγήκε:** καλύτερος στόχος 40% με −127.9k (ακριβώς −127,860). Στα ίδια seeds threshold −140.9k και fixed(22) −123.3k. Το 40% είναι εσωτερικό σημείο του grid, άρα η καμπύλη έχει πραγματικό ελάχιστο και όχι άκρη. **Τι σημαίνει:** το HPA είναι καλύτερο από τον threshold και λίγο χειρότερο από το fixed(22). Όμως ρυθμίστηκε και μετρήθηκε στα ίδια seeds (in-sample), άρα δεν είναι αποτέλεσμα αξιολόγησης.



Σχήμα Β. Rev-1 gate: return(threshold) μείον return(static_ramp) ανά seed. Όλες οι μπάρες είναι αρνητικές. Πηγή: `20261017-152657-R-20261017-013-e0-a1/preflight_rev1.json`.



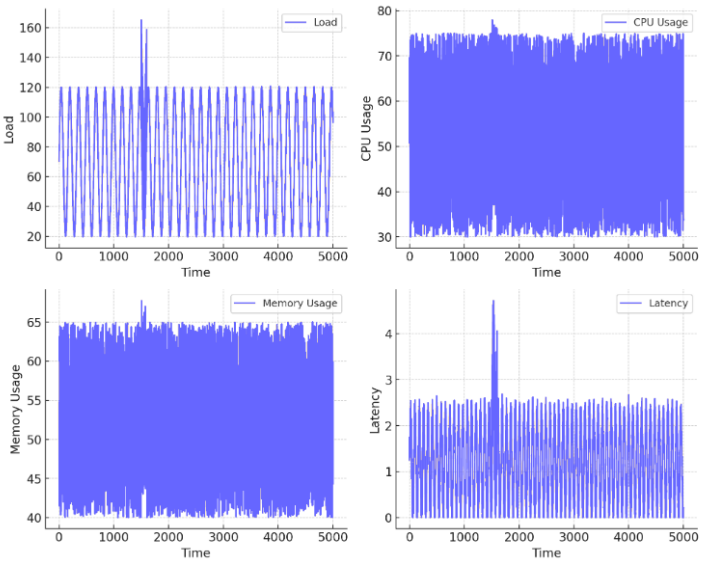
Σχήμα Γ. Mean return του K8sHPA ανά στόχο CPU%. Μόνο seeds ανάπτυξης. Πηγή: `code/experiments/dinos_simv2/hpa_selection_dev.json`.

5 Αντιστοιχία με τα σχήματα του DInos

Κάθε ενότητα δείχνει πρώτα το σχήμα του DInos και μετά το δικό μας. Τα Fig 1–3 του DInos είναι αρχιτεκτονικά διαγράμματα και δεν αναπαράγονται. Όπου γράφει «αναλογία», το δικό μας σχήμα δείχνει άλλο πράγμα από το αρχικό (π.χ. baseline αντί για agent). Οι agents (DERP, DInos, LSTM) δεν έχουν εκπαιδευτεί ακόμη. Το run W0 (R-20261017-015) τρέχει αυτή τη στιγμή.

DInos	Δικό μας	Κατάσταση
Fig 4	fig4	Αναλογία (η μνήμη λείπει, δείχνουμε backlog)
Fig 5	fig5	ΔΕΝ ΥΠΑΡΧΕΙ ακόμη (χωρίς agents), δείχνουμε baselines
Fig 6 αριστερά	fig6_left	Αναλογία: threshold αντί για DERP
Fig 6 δεξιά	fig6_right	Αναλογία: oracle αντί για DInos
Fig 7 αριστερά	fig7_left	Μερική αναλογία, χωρίς real cluster
Fig 7 δεξιά	fig7_right	Μερική αναλογία, trace προσομοιωτή
Fig 8	fig8	Μερικός πίνακας, μόνο baselines

5.1 Fig 4: μεγέθη εισόδου (φορτίο, CPU, μνήμη, latency, 5000 βήματα)

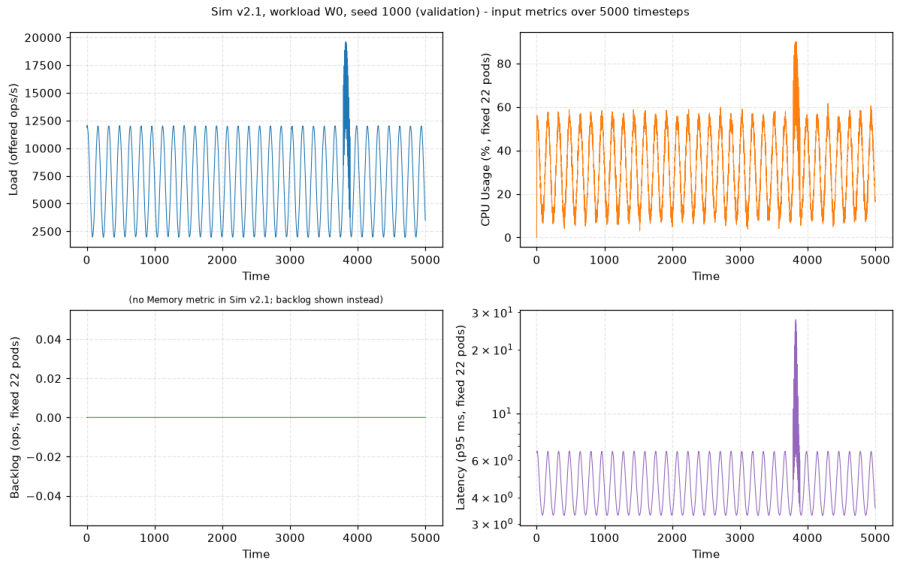


DInos, Fig 4: είσοδοι της προσομοίωσης σε 5000 βήματα.

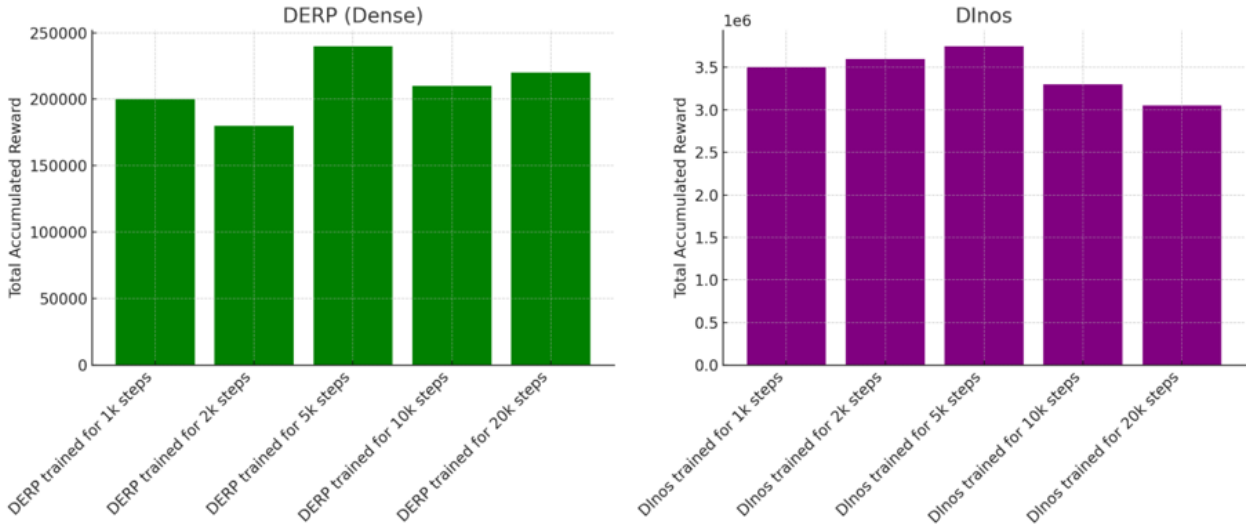
Βλέπουμε τα μεγέθη εισόδου του προσομοιωτή σε ένα επεισόδιο. Το φορτίο είναι ημίτονο με spike, όπως στο DInos. Στο DInos το spike είναι στα βήματα 1500–1600. Στο δικό μας W0 διαρκεί 100 βήματα σε τυχαία θέση ανά seed, και η φάση του ημιτόνου είναι επίσης τυχαία (`code/sim_v2/workloads.py`). Οι κλίμακες διαφέρουν (ops/s, ms) και δεν υπάρχει μνήμη. Η σύγκριση είναι μόνο ποιοτική.

5.2 Fig 5: reward ανά checkpoint εκπαίδευσης (1k–20k)

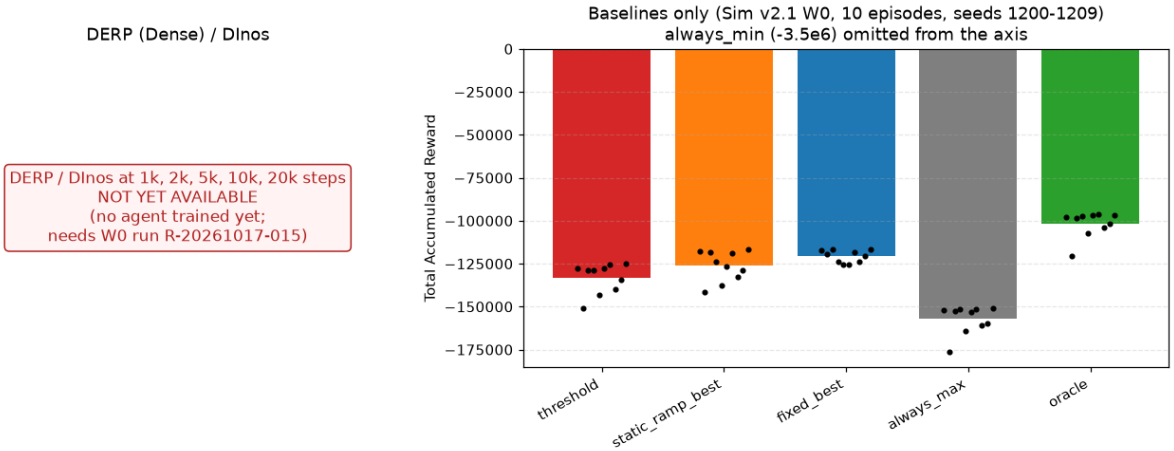
Το αριστερό πάνελ είναι κενό επειδή δεν έχουμε εκπαιδευμένους agents. Θα γεμίσει μετά το R-20261017-015. Δεξιά βλέπουμε το συνολικό reward των baselines: threshold −133.1k, static_ramp −126.2k, fixed_best −120.8k, always_max −157.2k, oracle −101.6k. Το δικό μας reward είναι αρνητικό (κόστος) ενώ στο DInos είναι θετικό, άρα οι τιμές δεν συγκρίνονται άμεσα.



Δικό μας (αναλογία): offered load, CPU, backlog και p95 latency με 22 σταθερά pods, seed 1000.

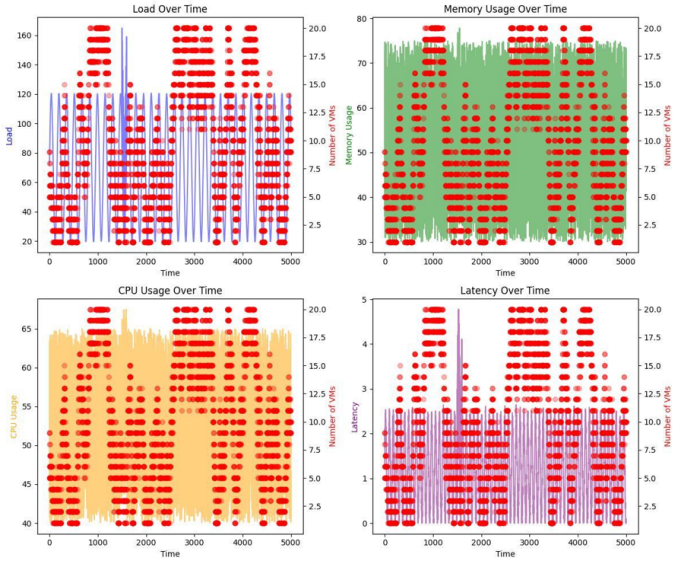


DInos, Fig 5: συνολικό reward των DERP (αριστερά) και DInos (δεξιά) στα 1k, 2k, 5k, 10k, 20k βήματα εκπαίδευσης.

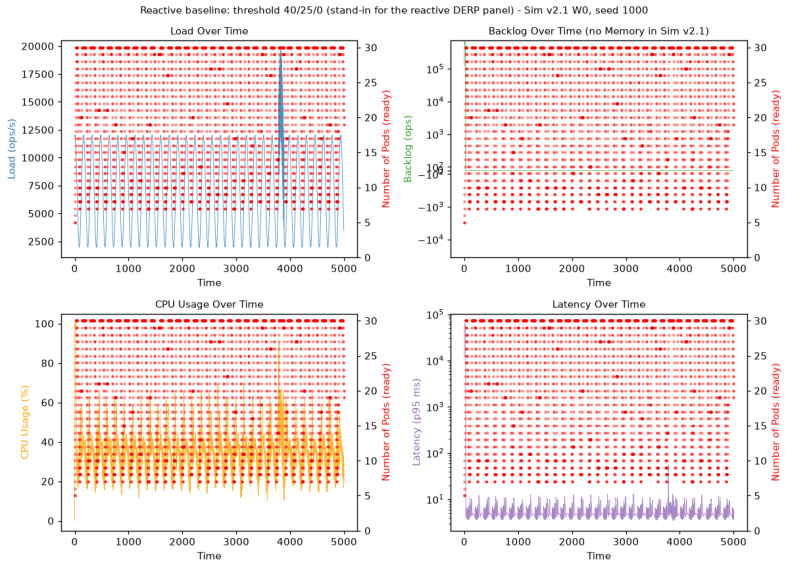


Δικό μας: ΔΕΝ ΥΠΑΡΧΕΙ ακόμη για agents. Δεξιά, το reward των baselines σε 10 επεισόδια (seeds 1200–1209).

5.3 Fig 6 αριστερά: αντιδραστικός ελεγκτής, μετρικές και pods



DInos, Fig 6 αριστερά: ο DERP αντιδρά μετά το γεγονός και κάνει υπερ- ή υπο-παροχή.



Δικό μας (αναλογία): ο threshold baseline, όχι DERP.

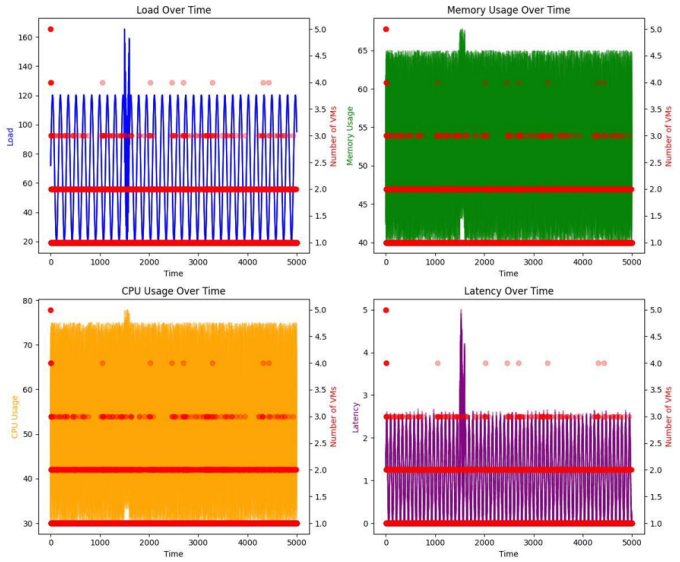
Ο threshold παίζει εδώ τον ρόλο του αντιδραστικού ελεγκτή. Ανεβαίνει στο μέγιστο των 30 pods στην εκκίνηση και μετά ταλαντεύεται γύρω από το φορτίο. Η αρχική φάση κοστίζει λόγω της ποινής SLA. Το σχήμα δεν είναι αποτέλεσμα DERP.

5.4 Fig 6 δεξιά: ελεγκτής που προβλέπει

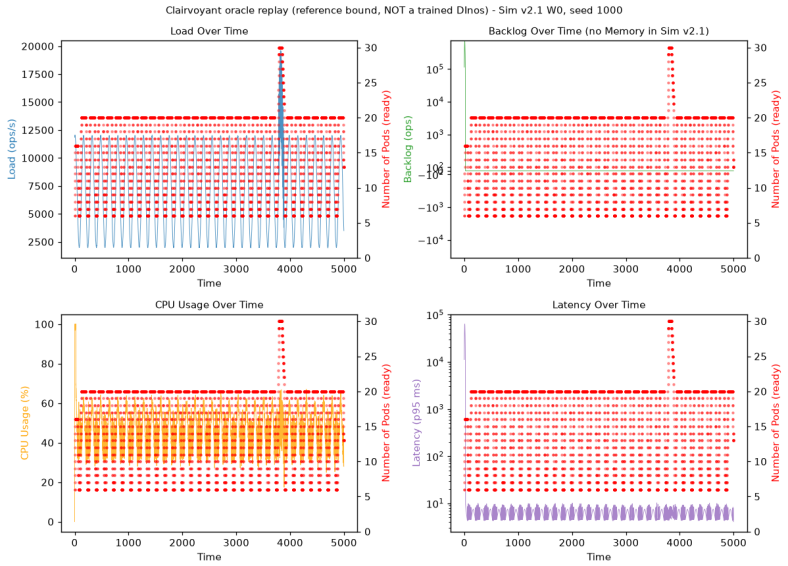
Ο oracle χρησιμοποιεί λιγότερα pods (μέσος όρος 15.9 έναντι 22.2 του threshold) και έχει return −108.4k (seeds 1000–1004) έναντι −140.9k. Δείχνει πόσο περιθώριο υπάρχει. Δεν δείχνει ότι το DInos θα το πετύχει.

5.5 Fig 7 αριστερά: αθροιστικό reward

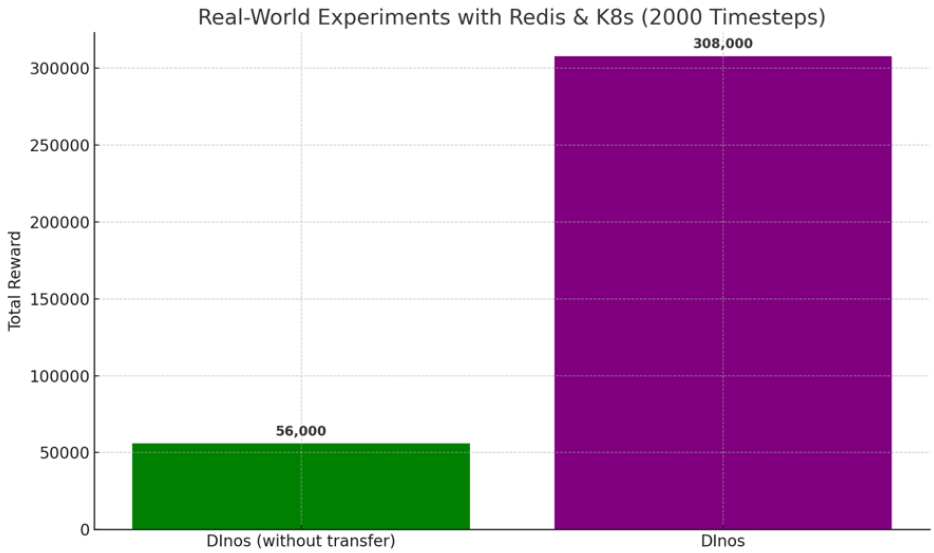
Real-cluster run αυτού του τύπου δεν έχουμε. Τελικές τιμές: threshold −140,927, fixed(22) −123,345, oracle −108,446. Ο threshold ξεκινά από 5–6 pods και το fixed(22) είναι προ-παρασχεμένο. Έτσι τα



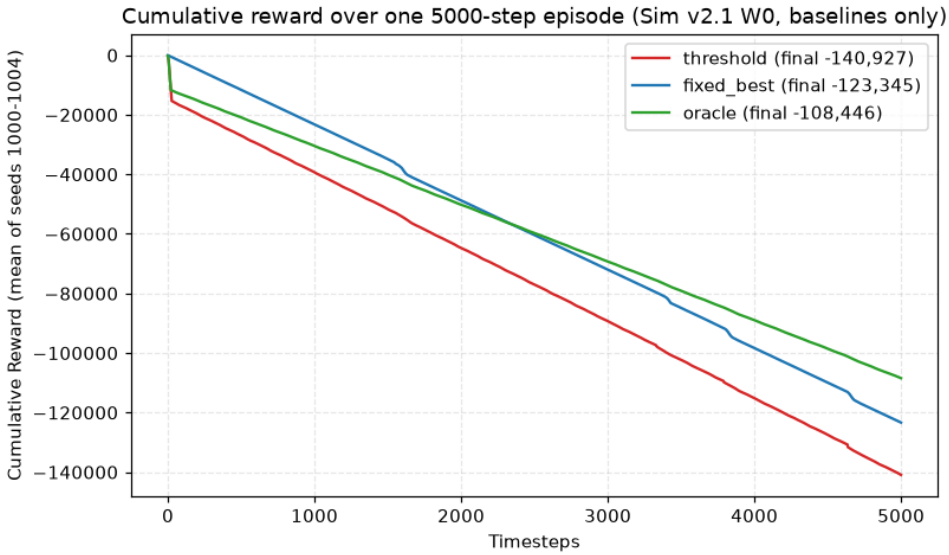
DInos, Fig 6 δεξιά: ο DInos προβλέπει το φορτίο.



Δικό μας (αναλογία): ο oracle, που ξέρει το μέλλον, όχι DInos.



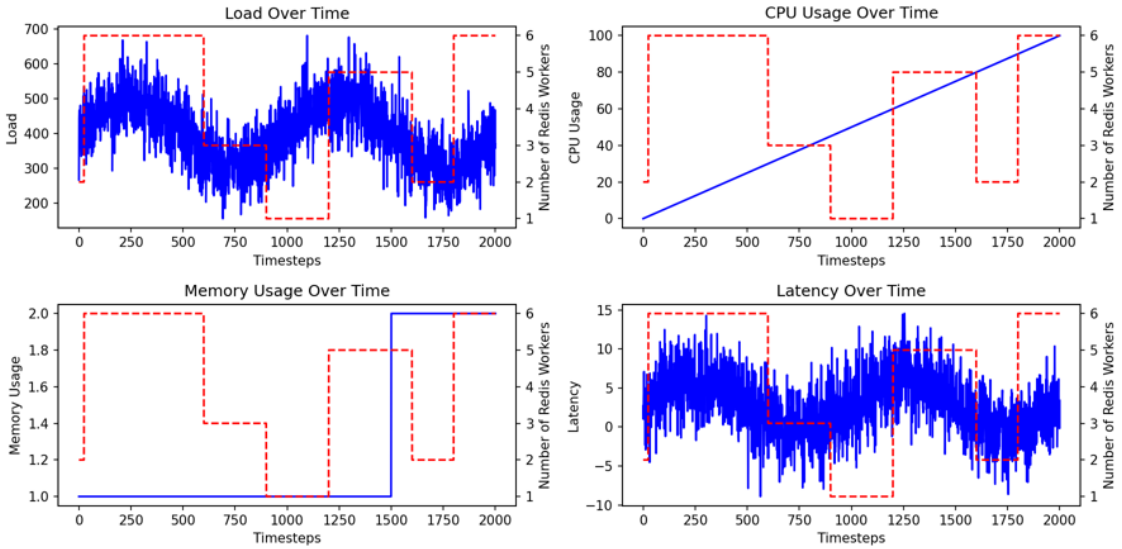
DInos, Fig 7 αριστερά: πραγματικό cluster, αθροιστικό reward σε 2000 βήματα, DInos 308,000 έναντι LSTM-only 56,000.



Δικό μας (μερική αναλογία): αθροιστικό reward baselines στον προσομοιωτή, μέσος όρος seeds 1000–1004, 5000 βήματα.

πρώτα 150 βήματα εξηγούν ένα μέρος της διαφοράς (startup: threshold −18.4k, fixed −3.5k).

5.6 Fig 7 δεξιά: μετρικές και pods στον χρόνο

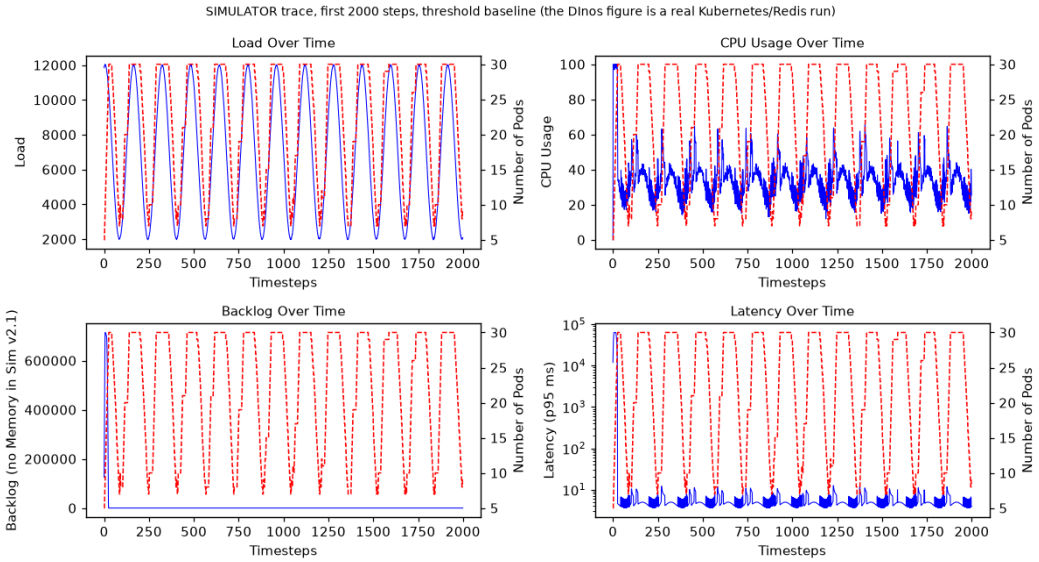


DInos, Fig 7 δεξιά: πραγματικό cluster, φορτίο, CPU, μνήμη, latency και αριθμός Redis pods σε 2000 βήματα.

Δεν είναι πραγματική μέτρηση και δεν υπάρχει agent. Το σχήμα δείχνει μόνο ότι μπορούμε να παράγουμε διάγραμμα στο ίδιο στυλ. Οι calibration runs του Οκτωβρίου μετρούν το cluster, δεν τρέχουν autoscaler.

5.7 Fig 8: πίνακας σύνοψης

Οι γραμμές DERP, DInos και real cluster είναι κενές (not available). Η μνήμη δεν υπάρχει στον προσομοιωτή. Το CPU είναι το παρατηρούμενο, με θόρυβο. Το p95 του threshold κυριαρχείται από την εκκίνηση (αρχικό backlog). Τα threshold και fixed(22) επιλέχθηκαν στα ίδια seeds, άρα είναι in-sample.



Δικό μας (μερική αναλογία): τα πρώτα 2000 βήματα ενός trace του προσομοιωτή (threshold), μπλε γραμμή και κόκκινα διακεκομμένα pods.

Method	Environment	Results (Accumulated Reward)
DERP	Simulation	≈ 220,000
DInos (with transfer)	Simulation	≈ 3,800,000
DInos (without transfer)	Real World	≈ 56,000
DInos (with transfer)	Real World	≈ 308,000

DInos, Fig 8: πίνακας DERP εναντίον DInos σε προσομοίωση και πραγματικό κόσμο (reward 220k έναντι 3.8M, 56k έναντι 308k).

Method	Environment	Accumulated Reward	mean CPU %	mean p95 ms	mean pods
DERP	Simulation (Sim v2.1)	not yet available	-	-	-
DInos (with transfer)	Simulation (Sim v2.1)	not yet available	-	-	-
threshold (baseline)	Simulation (Sim v2.1)	-140,927	34.9	126.1	22.2
fixed_best (baseline)	Simulation (Sim v2.1)	-123,345	32.3	4.8	22.0
oracle (baseline)	Simulation (Sim v2.1)	-108,446	45.2	79.7	15.9
any agent	Real cluster	not available	-	-	-

Δικό μας: μερικός πίνακας, μόνο baselines στον προσομοιωτή (seeds 1000–1004, μέσος όρος).

Μέθοδος (sim v2.1)	Reward	CPU %	p95 ms	Pods
DERP		όχι ακόμη διαθέσιμο		
DInos (με transfer)		όχι ακόμη διαθέσιμο		
threshold	−140,927	34.9	126.1	22.2
fixed_best	−123,345	32.3	4.8	22.0
oracle	−108,446	45.2	79.7	15.9
Agent σε real cluster		δεν υπάρχει		

6 Τι σημαίνουν όλα αυτά και τι ακολουθεί

Τι έχουμε. Δύο gates απέτυχαν. Ο threshold χάνει από απλούς σταθερούς ελεγκτές, τόσο με άδικη όσο και με δίκαιη εκκίνηση. Η κατάταξη αλλάζει όταν αλλάζουμε παραμέτρους που δεν έχουμε μετρήσει. Ο oracle δείχνει ότι υπάρχει περιθώριο βελτίωσης (−101.6k έναντι −126.2k για το static_ramp).

Τι ακολουθεί.

- **W0 (R-20261017-015).** Το εξερευνητικό run με τους agents (`dinos_t`, `lstm_scratch`, `derp`, 3 training seeds, 100,000 βήματα, τελευταίο checkpoint) τρέχει τώρα. Ό,τι βγει θα είναι διερευνητικό. Θα γεμίσει το Fig 5 και τον πίνακα του Fig 8.
- **Βαθμονόμηση.** Θα μετρήσουμε στο πραγματικό cluster τις παραμέτρους δ, k, ϑ και d (startup delay), ώστε ο προσομοιωτής να μη στηρίζεται σε εικασίες. Μόνο μετά μπορούμε να ισχυριστούμε κάτι επιβεβαιωτικό, με νέο παγωμένο πείραμα στα seeds 20000+.

Ειλικρινή όρια.

- Όλα είναι **exploratory**. Το W0 επιτράπηκε ως διερεύνηση META τα FAIL, και δεν τα διορθώνει.
- Οι gates **απέτυχαν** και δεν ξανατρέχουν με άλλους κανόνες.
- Ο προσομοιωτής **δεν είναι βαθμονομημένος**.
- Τα traces των seeds 1000–1004 είναι **in-sample**: τα threshold, fixed(22) και HPA ρυθμίστηκαν εκεί.
- Το HPA είναι post-hoc baseline (προστέθηκε μετά τα FAIL) και στηρίζεται στην υπόθεση request = limit. Μέρη της προδιαγραφής του (`docs/hpa_spec.md`) είναι παράφραση του κώδικα και δεν έχουν ξαναελεγχθεί στο raw αρχείο.
- Τα σχήματα φτιάχτηκαν από αποθηκευμένα traces και returns (`make_report_figs.py`). Δεν έτρεξε προσομοιωτής ή εκπαίδευση. Το static_ramp έχει μόνο returns, όχι traces ανά βήμα.

Γλωσσάρι

Seed
Αριθμός που καθορίζει την τυχαιότητα ενός επεισοδίου. Ίδιο seed, ίδιο φορτίο.

Επεισόδιο
Μία πλήρης εκτέλεση 5000 βημάτων των 15 δευτερολέπτων.

Gate
Έλεγχος που πρέπει να περάσει ο προσομοιωτής πριν εμπιστευτούμε συγκρίσεις agents.

CI (διάστημα εμπιστοσύνης)
Περιοχή τιμών μέσα στην οποία πέφτει πιθανότατα η πραγματική μέση διαφορά. Εδώ 95%, υπολογισμένο με bootstrap (επαναληπτική δειγματοληψία των δεδομένων).

Baseline
Απλός ελεγκτής για σύγκριση.

Oracle
Ελεγκτής που ξέρει το μέλλον. Δεν εφαρμόζεται στην πράξη και δείχνει το όριο.

p95 latency
Χρόνος απόκρισης που το 95% των αιτημάτων τον ξεπερνά μόνο σε 5% των περιπτώσεων.

SLA
Συμφωνημένο όριο ποιότητας, εδώ όριο στο latency.

In-sample
Μετρημένο στα ίδια δεδομένα όπου ρυθμίστηκε η μέθοδος. Δείχνει υπερβολικά καλά.

Exploratory
Διερευνητικό: δεν είναι αποδεικτικό αποτέλεσμα.

Βαθμονόμηση
Ρύθμιση παραμέτρων του προσομοιωτή από μετρήσεις του πραγματικού cluster.

HPA
Horizontal Pod Autoscaler, ο ενσωματωμένος αυτόματος scaler του Kubernetes.

Preflight
Ο πρώτος έλεγχος (gate) πριν από όλα τα άλλα.